PAGE NO.: 31 | 8 | 26

# Number System

→ Number is a Math value used for counting or measuring or labelling of .

→ Number System is defined as a system of writing to express numbers. It is ~~one~~ the mathematical notation for representing no. of a given set by using digits.

Following are some N.S:- i) Decimal N.S.
ii) Binary N.S.
iii) Octal N.S.
iv) Hexadecimal N.S

I) Decimal Number System:- No. used in day-to-day life.

Base = 10 ⇒ (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

eg:- $2586 = (2\times10^3) + (5\times10^2) + (8\times10^1) + (6\times10^0)$
$= 2000 + 500 + 80 + 6$
$= 2586$

II) Binary N.S:- Widely used in Computers. All inputs given to a Comp. are 1st converted to BNS b/c it understand only B.N.S.

Base = 2 ⇒ (0, 1)

eg:- $10101 = (1\times2^4) + (0\times2^3) + (1\times2^2) + (0\times2^1) + (1\times2^0)$
$= 16 + 0 + 4 + 0 + 1$
$= \boxed{(21)_{10} = (10101)_2}$

$(10100011)_2 = 128 + 0 + 32 + 0 + 0 + 0 + 2 + 1$
$= (163)_{10}$

III) Octal N.S.:- Base ⇒ 8 (0, 1, 2, 3, 4, 5, 6, 7)

eg:- $(2057)_8$ = 1024 + 0 + 40 + 7

$8^3\ 8^2\ 8^1\ 8^0$ = $(1071)_{10}$

512 64 8 1

eg:- $(2671)_8$ = 1024 + 384 + 56 + 1

= $(1465)_{10}$

eg:- $(2786)_8$ = 128 + 64 + 6

invalid = $(198)_{10}$ ✗

IV) Hexadecimal N.S.:- Base ⇒ 16 (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

16 Symbols or Character (Single)

10 - A, 11 - B, 12 - C, 13 - D, 14 - E, 15 - F

$(1AF)_{16}$ = 256 + 160 + 15

= $(431)_{10}$

$(7CF)_{16}$ = 1792 + 192 + 15

= $(1999)_{10}$

| $2^0$ | $2^1$ | $2^2$ | $2^3$ | $2^4$ | $2^5$ | $2^6$ | $2^7$ | $2^8$ | $2^9$ | $2^{10}$ |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 4 | 8 | 16 | 32 | 64 | 128 | 256 | 512 | 1024 |

| $2^{11}$ | $2^{12}$ | $8^0$ | $8^1$ | $8^2$ | $8^3$ | $8^4$ |
|---|---|---|---|---|---|---|
| 2048 | 4096 | 1 | 8 | 64 | 512 | 4096 |

| $16^0$ | $16^1$ | $16^2$ | $16^3$ | $16^4$ |
|---|---|---|---|---|
| 1 | 16 | 256 | 4096 | 65536 |

PAGE NO.:

# Converting from Decimal to Binary :-

$(25)_{10} \Rightarrow$

| | | |
|---|---|---|
| 2 | 25 | 1 |
| 2 | 12 | 0 |
| 2 | 6 | 0 |
| 2 | 3 | 1 |
| | 1 | |

$\Rightarrow (11001)_2 \Rightarrow 16+8+0+0+1$
$= (25)_{10}$

$(42)_{10} \Rightarrow$

| | | |
|---|---|---|
| 2 | 42 | 0 |
| 2 | 21 | 1 |
| 2 | 10 | 0 |
| 2 | 5 | 1 |
| 2 | 2 | 0 |
| | 1 | |

$\Rightarrow (101010)_2$

$(156)_{10} \Rightarrow$

| | | |
|---|---|---|
| 2 | 156 | 0 |
| 2 | 78 | 0 |
| 2 | 39 | 1 |
| 2 | 19 | 1 |
| 2 | 9 | 1 |
| 2 | 4 | 0 |
| 2 | 2 | 0 |
| | 1 | |

$\Rightarrow (10011100)_2$

# Decimal to Octal :-

$(952)_{10} =$

| | | |
|---|---|---|
| 8 | 952 | 0 |
| 8 | 119 | 7 |
| 8 | 14 | 6 |
| | 1 | |

$\Rightarrow (1670)_8$

$(210)_{10} =$

| | | |
|---|---|---|
| 8 | 210 | 2 |
| 8 | 26 | 2 |
| | 3 | |

$\Rightarrow (322)_8$

# Converting Decimal to Hexadecimal :-

$(428)_{10}$ = 16 | 428 | 12 ⇒ $(1AC)_{16}$
16 | 26 | 10
1

$(188)_{10}$ = 16 | 188 | 12 ⇒ $(BC)_{16}$
11

# Converting from a base other than 10 to to base other than 10.

1) Convert the original no. to decimal no.
2) Convert the Decimal no. to req. no. (with new base).

eg- $(101110)_2$ = $(\ )_8$ ⇒ 32 + 8 + 4 + 2
⇒ $(46)_{10}$

8 | 46 | 6 ⇒ $(56)_8$
5

$(1AB)_{16}$ = 256 + 160 + 11
= $(427)_{10}$

| | | |
|---|---|---|
| 2 | 427 | 1 |
| 2 | 213 | 1 |
| 2 | 106 | 0 |
| 2 | 53 | 1 |
| 2 | 26 | 0 |
| 2 | 13 | 1 |
| 2 | 6 | 0 |
| 2 | 3 | 1 |
| | 1 | |

⇒ $(110101011)_2$

$(11010011)_2$ = 128 + 64 + 16 + 2 + 1
= $(211)_{10}$

16 | 211 | 3 ⇒ $(D3)_{16}$
13

# Shortcut from Binary to Octal

1) Divide Binary digit into group of 3 digits (Starting from right)
2) Convert each group of 3 Binary Digit into one octal digits.

| Decimal | HexaDecimal | Binary | Octal |
|---|---|---|---|
| 0 | 0 | 0000 | 0 |
| 1 | 1 | 0001 | 1 |
| 2 | 2 | 0010 | 2 |
| 3 | 3 | 0011 | 3 |
| 4 | 4 | 0100 | 4 |
| 5 | 5 | 0101 | 5 |
| 6 | 6 | 0110 | 6 |
| 7 | 7 | 0111 | 7 |
| 8 | 8 | 1000 | |
| 9 | 9 | 1001 | |
| 10 | A | 1010 | |
| 11 | B | 1011 | |
| 12 | C | 1100 | |
| 13 | D | 1101 | |
| 14 | E | 1110 | |
| 15 | F | 1111 | |

eg:- $(\underset{5}{101}\underset{6}{110})_2 \rightarrow (56)_8 \Rightarrow (56)_8$

$(\underset{1}{1}\underset{5}{101}\underset{2}{010})_2 \rightarrow (152)_8$

PAGE NO.:

# Short cut for Octal to Binary:-

1) Convert each octal digit to 3 digit Binary Number.
2) Combine all resulting Binary groups into Single B. Number

$(562)_8 = (101\ 110\ 010)_2$ ; $6751 = 110\ 111\ 101\ 001$

# Short for Binary to Hexadecimal:-

1) Convert each H.D. to 4 digit B. Number.
2) Combine all grps together.

$\underbrace{(1110}_{1\ D\ 13}\ \underbrace{1111)}_{F\ 15}{}_2 = (1DF)_{16}$

$\underbrace{111}_{7}\ \underbrace{0010}_{2}\ \underbrace{1011}_{11} = (72B)_{16}$

# Short for Hexadecimal to Binary:-

$(2AB)_{16} = (0010\ 1010\ 1011)_2$

$(ABC)_{16} = (1010\ 1011\ 1110)_2$

# Fractional no. Conversation:- [① First Fraction to Decimal]

$\underbrace{(110}_{4\,2\,1}\ \underbrace{.101)_2}_{\frac{1}{2}\ 0\ \frac{1}{8}}$ $\overset{-1/2\ -1/8}{}$

$\Rightarrow 6 + \frac{5}{8} \Rightarrow 6 + 0.625$

$\Rightarrow \boxed{6.625}_{10}$

$(127.54)_8 = 64 + 16 + 7 + \frac{5}{8} + \frac{4}{64} \Rightarrow \boxed{87.6875}_{10}$

$$(2B.C4)_{16} = 32 + 11 + \frac{12}{16} + \frac{4}{256} = (43.765625)_{10}$$

$(0.6875)_{10} = (?)_2$

| Base | | Fraction | integer |
|---|---|---|---|
| $0.6875 \times 2$ | = | 0.3750 | 1 |
| $0.3750 \times 2$ | = | 0.7500 | 0 |
| $0.7500 \times 2$ | = | 0.5000 | 1 |
| $0.5000 \times 2$ | = | 0.0000 | 1 |

$= (0.1011)_2$

## Binary Arithmetic :-

**Add$^n$ :-**

$0 + 0 = 0$

$0 + 1 = 1$

$1 + 0 = 1$

$1 + 1 = 0$ (and 1 carry to next higher column.

| | | |
|---|---|---|
| 10011 | 10001 | 101101 |
| 01001 | 11101 | 11001 |
| 11100 | 101110 | 1000110 |

**Sub$^n$ :-**

$0 - 0 = 0$

$1 - 0 = 1$

$1 - 1 = 0$

$0 - 1 = 1$ (Borrow from next column)

| | | |
|---|---|---|
| 10110 11 | 10101 | 10000101 |
| 10016 | 100111 | 101100 |
| 1001001 | 000110 | 0001001 |

**mult-**

$0 * 0 = 0$
$0 * 1 = 0$
$1 * 0 = 0$
$1 * 1 = 1$

$$\begin{array}{r} 1101 \\ \times\ 100 \\ \hline 0000 \\ 0000\times \\ 1101\times\times \\ \hline 110100 \end{array}$$

$$\begin{array}{r} 1010 \\ \times\ 101 \\ \hline 1010 \\ 0000\times \\ 1010\times\times \\ \hline 110010 \end{array}$$

**Div-**

$0 / 0 =$ undefined
$0 / 1 = 0$
$1 / 0 =$ undefined
$1 / 1 = 1$

---

BCD → Binary Coded Decimal → It is a method of representing Decimal no. in Binary, where each Decimal Digit is stored seperately using 4 bit Binary code. Each D. Digit is converted into its corresponding 4 bit B. code.
Every digit is encoded seperately instead of converting the entire D. no into Binary.

| D. No | BCD |
|---|---|
| 0 | 0000 |
| 1 | 0001 |
| 2 | 0010 |
| 3 | 0011 |
| 4 | 0100 |
| 5 | 0101 |
| 6 | 0110 |
| 7 | 0111 |
| 8 | 1000 |
| 9 | 1001 |

eg- 57 ⇒ $(0101\,0111)_{BCD}$
(0101 → 5, 0111 → 7; BCD)

1 Byte = 8 bit
1 nibble = 4 bit = $\frac{1}{2}$ Byte

→ Packed BCD :- It stores 2 Digit in a single 8-bit; each 4 bit (nibble) represents 1 D. digit.

eg :- 9 → 1001
3 → 0011
$\Rightarrow (93)_{10} \Rightarrow (10010011)_{BCD}$

→ Unpacked BCD :- It stores 1 D. digit per byte.

eg :- 9 → 0000 1001
3 → 0000 0011
$\Rightarrow (93)_{10} \Rightarrow (00001001\ 00000011)_{BCD}$

→ BCD-Addition :- To add 2 BCD no., add corresponding 4-bit groups as normal binary numbers.
If the sum of any 4-bit group is > than 1001 (9), then 0110 (6) is added as a correction to obtain a valid BCD.

eg :-
```
  0010          23        0010  0011
  0101          45      + 0100  0101
  0111          68        0110  1000
                          (6)   (8)
```

```
45  0100  0101
37  0011  0111
    0111  1100
          0110
92 → 1000  0010
```

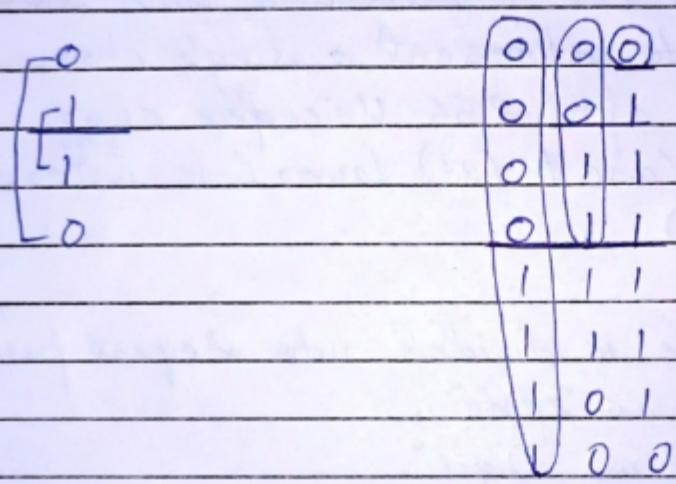
→ BCD - Substraction - To substract 2 BCD nos, Substract the corresponding BCD Digits, if the result is not a valid BCD Digit, a correction factor (0110)(6) is applied

| | | |
|---|---|---|
| 47 | 0100 | 0111 |
| 23 | 0010 | 0011 |
| 24 | 00 0 | 0100 |

→ XS-3 Code:- It is a non-weighted code, used to express decimal nos.
This code is derived from BCD code by adding $(0011)_2$ or $(3)_{10}$ to each BCD code

| | | |
|---|---|---|
| 0 | 0000 | 0011 |
| 1 | 0001 | 0100 |
| 2 | 0010 | 0101 |
| 3 | 0011 | 0110 |
| 4 | 0100 | 0111 |
| 5 | 0101 | 1000 |
| 6 | 0110 | 1001 |
| 7 | 0111 | 1010 |
| 8 | 1000 | 1011 |
| 9 | 1001 | 1100 |

→ Gray Code :- It is a non weighted code, and its not arithmetic code that means there are no specific weight assigned to the bit position, it has a very special feature that has only one bit change, each time the decimal no. is increment as only 1 bit change at a time, It is also called as unit distance code. Used in K-map.



→ ASCII Code :- Bits can also be coded to represent Alphabets, Numbers & Sp. character.
- American Std. Code for Info. Interchange.
- $2^7 = 128$ Char.

"A" is → 65 ; "Z" → 90
"a" is → 97 ; "z" → 122
"0" is → 48 ; "9" → 57

→ EBCDIC Code:-
↳ Extended Binary Coded Decimal Interchange Code.
⇒ An 8-bit char. encoding system developed by IBM in 1963-64, for their mainframe & mid-range Comp. systems.
⇒ It translates human readable char. like letters, no. & Sp. symbols into B. Code, so that large Comp. Processors can understand and store them.
⇒ It uses 1 byte to represent a single char.
⇒ It can represent upto 256 Unicode char. (Including Upper Case & ($2^3$) lower case letters, nos, & Sp. char.)

⇒ Each 8-bit code is divided into 2 equal parts:-
1) 1st 4 bits act as "zone",
2) Las 4 bits act as "Digit".

PAGE NO.:

# Boolean Algebra:- It deals with the B.no. System, the variable used in the Boolean eq" may assume only 2 possible value (0 & 1).

→ Logical Addition:- Symbol "+" is used for L.A. operator.
→ It is called "OR" operator.

→ Truth table for the Logical "OR" operator is:-

| Input | | Output |
|---|---|---|
| A + B | = | C |
| 0 + 0 | = | 0 |
| 0 + 1 | = | 1 |
| 1 + 0 | = | 1 |
| 1 + 1 | = | 1 |

→ Logical Multiple:- Symbol "." used for multiplication operator.
→ Also called "AND" operator.

→ Truth table for the Logical "AND" operator is:-

| Input | | Output |
|---|---|---|
| A . B | = | C |
| 0 . 0 | = | 0 |
| 0 . 1 | = | 0 |
| 1 . 0 | = | 0 |
| 1 . 1 | = | 1 |

⇒ Complementation:- It is a uniary operation which is defined on a single variable.

→ Symbol = "¯" is

→ It is normally used for Complementation operator.

→ Also known as "NOT" operator

→ Thus we write $\bar{A}$. meaning take complement of A.

| Input | Output |
|---|---|
| A | $\bar{A}$ |
| 0 | 1 |
| 1 | 0 |

⇒ Operator Precedance:-

1) Evaluation L → R
2) Parenthesis "()" are first evaluated.
3) All Compliment operation are performed next (NOT)
4) All [AND (.)]
5) All [OR (+)].

#⇒ Postulates of Boolean Algebra:-

1) a) A = 0 ; if and only if A is not equal to 1.
   b) A = 1 ; ∴ ∴ ∴ A ≠ 0.

2) a) $x + 0 = x$
   b) $x \cdot 1 = x$

3) Commutative Law :- a) $x + y = y + x$
   b) $x \cdot y = y \cdot x$

4) Associative Laws :- a) $x + (y+z) = (x+y) + z$

b) $x \cdot (y \cdot z) = (x \cdot y) \cdot z$

5) Distributive Laws :- a) $x \cdot (y+z) = x \cdot y + x \cdot z$

b) $x + (y \cdot z) = (x+y) \cdot (x+z)$

6) $x + \bar{x} = 1$

$x \cdot \bar{x} = 0$

⇒ The Principle of Duality :-

In Boolean Algebra, there is a precise duality b/w operators (.) AND, (+) OR and digits "0" and "1". The implication of this duality is that any theorem in Boolean Algebra has dual obtainable by interchanging (+) with (.) & "0" with "1", thus if a particular theorem is proved, it dual theorem automatically holds and need not to be proved seperately.

1) Theorem - 1 :- $x + x = x$

LHS → $x + x$

$(x+x) \cdot 1$ $[x \cdot 1 = x]$

$(x+x) \cdot (x+\bar{x})$ $[x + \bar{x} = 1]$

$x + x \cdot \bar{x}$ $[x + y \cdot z = (x+y) \cdot (x+z)]$

$x + 0$ $[x \cdot \bar{x} = 0]$

RHS → $x$

PAGE NO.:

## Theorem-2:- $x + 1 = 1$

LHS → $x + 1$

$(x+1) \cdot 1$ $[x \cdot 1 = x]$

$(x+1) \cdot (x + \bar{x})$ $[x + \bar{x} = 1]$

$x + 1 \cdot \bar{x}$ $[x + y \cdot z = (x+y) \cdot (x+z)]$

$x + \bar{x} \cdot 1$ $[x \cdot y = y \cdot x]$

$x + \bar{x}$ $[x \cdot 1 = x]$ $[x + \bar{x} = 1]$

RHS → 1

## Theorem-3:- $x + x \cdot y = x$

LHS → $x + x \cdot y$

$x \cdot 1 + x \cdot y$ $[x \cdot 1 = x]$

$x \cdot (1 + y)$ $[x \cdot (y+z) = x \cdot y + x \cdot z]$

$x \cdot (y + 1)$ $[x + y = y + x]$

$x \cdot 1$ $[x + 1 = 1]$

RHS → $x$ $[x \cdot 1 = x]$

→ The theorem of Boolean Algebra can also be proved by the Truth Table

⇒ $x + x \cdot y = x$

equal (=)

| $x$ | $y$ | $x \cdot y$ | $x + x \cdot y$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 |

⇒ $x \cdot (x+y) = x$

equal(=)

| $x$ | $y$ | $x+y$ | $x \cdot (x+y)$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 |

⇒ $\bar{\bar{x}} = x$

equal(=)

| $x$ | $\bar{x}$ | $\bar{\bar{x}}$ |
|---|---|---|
| 0 | 1 | 0 |
| 1 | 0 | 1 |

⇒ $x \cdot (\bar{x}+y) = x \cdot y$

equal(=)

| $x$ | $y$ | $\bar{x}$ | $\bar{x}+y$ | $x \cdot (\bar{x}+y)$ | $x \cdot y$ |
|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 1 | 1 |

⇒ $x + \bar{x} \cdot y = x+y$

equal(=)

| $x$ | $y$ | $\bar{x}$ | $\bar{x} \cdot y$ | $x + \bar{x} \cdot y$ | $x+y$ |
|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 | 1 | 1 |

# De-Morgan's Law :-

→ De Morgan (a mathematician) developed a pair of important rules regarding group complimentation (compliment of a group of terms) in Boolean Algebra. These theorems state that "AND" and "OR" operators are interchangeable through negation.

I) Theorem-1 :- The compliment of the sum of 2 or more variable is equal to the product of the compliment of the variables for two variables 'A' and 'B', the theorem is written in boolean notation as follows :-

$$\overline{A+B} = \bar{A} \cdot \bar{B}$$

equal (=)

| A | B | $\bar{A}$ | $\bar{B}$ | $\bar{A} \cdot \bar{B}$ | $A+B$ | $\overline{A+B}$ |
|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 | 0 | 1 |
| 0 | 1 | 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 0 | 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 | 0 | 1 | 0 |

II) Theorem-2 :- The compliment of the product of two or more variables is equal to the sum of the compliment of the variables for two variable 'x' and 'y', this theorem is written as in boolean Algebra.

$$\overline{x \cdot y} = \bar{x} \cdot \bar{y}.$$

equal (=)

| x | y | $x \cdot y$ | $\overline{x \cdot y}$ | $\bar{x}$ | $\bar{y}$ | $\bar{x} + \bar{y}$ |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 0 | 0 | 0 |

→ The generalised form of De-Morgan's Theorem:-

$$\overline{x_1 + x_2 + x_3 + \dots x_n} = \bar{x}_1 \cdot \bar{x}_2 \cdot \bar{x}_3 \cdot \bar{x}_4 \dots \bar{x}_n$$

$$\overline{x_1 \cdot x_2 \cdot x_3 \dots x_n} = \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \dots \bar{x}_n$$

→ Boolean func" :- A Boolean func" is an expression formed with Binary variables, the 2 Binary Operators "OR" & "AND", the uniary operator "NOT", Parenthesis and equal sign.

$$w = x \cdot \bar{y} \cdot z$$

Here, the var. $w$ is a func" of $x$, $y$ & $z$,

Written as :- $w = F(x, y, z)$.

| x | y | z | w |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

→ It is possible to find 2 Algebraic expressions that specify the same boolean func".

eg:- Consider these 2 eq"s:-

1) $F_1 = \bar{x} \cdot \bar{y} \cdot z + \bar{x} \cdot y \cdot z + x \cdot \bar{y}$
2) $F_2 = x \cdot \bar{y} + \bar{x} \cdot z$

| x | y | z | $F_1$ | $F_2$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 1 | 0 | 0 |

→ Need of Reduction:- When a boolean func" is implemented with logic gates, each var. in the func" designates an input to a gate an each term is implemented with a gate, thus for a given Boolean func", the minimization of the no. of var. and the no of terms will result in a less equipments, for minimizing the no. of var. in a given boolean func" we will use the method of Algebraic manipulation

eg:- a) $x + \bar{x} \cdot \bar{y} \Rightarrow (x+\bar{x}) \cdot (x+y)$ $[x + y \cdot z = (x+y) \cdot (x+z)]$
$= 1 \cdot (x+y)$ $[x + \bar{x} = 1]$

b) $x(\bar{x}+y) \Rightarrow x \cdot \bar{x} + x \cdot y$ $[x \cdot (y+z) = x \cdot y + x \cdot z]$
$= 0 + x \cdot y$ $[x \cdot \bar{x} = 0]$
$= x \cdot y$

c) $\bar{x} \cdot \bar{y} \cdot z + \bar{x} \cdot y \cdot z + x \cdot \bar{y}$

$\bar{x} \cdot z (\bar{y} + y) + x \cdot \bar{y}$ $\quad [x \cdot (y+z) = x \cdot y + x \cdot z]$

$x \cdot z \cdot 1 + x \cdot \bar{y}$ $\quad [x + \bar{x} = 1]$

$\bar{x} \cdot z + x \cdot \bar{y}$

→ Complement of a func$^n$ :- the comp. of a func "F" is "$\bar{F}$" and it is obtained by interchanging 0 for 1 and 1 for 0 in the truth table

$F = x \cdot \bar{y} + \bar{x} \cdot z$

| | x | y | z | F | $\bar{F}$ | $\bar{x}$ | $\bar{y}$ | $x \cdot \bar{y}$ | $\bar{x} \cdot z$ |
|---|---|---|---|---|---|---|---|---|---|
| | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 0 + 0 = 0 | |
| = | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 0 + 1 = 1 | |
| | 0 | 1 | 0 | 0 | 1 | 1 | 0 | 0 + 0 = 0 | |
| = | 0 | 1 | 1 | 1 | 0 | 1 | 0 | 0 + 1 = 1 | |
| = | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 1 + 0 = 1 | |
| = | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 1 + 0 = 1 | |
| | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 0 + 0 = 0 | |
| | 1 | 1 | 1 | 0 | 1 | 0 | 0 | 0 + 0 = 0 | |

* Algebraically, the comp. of a func$^n$ may be derived through De-Morgan's Theorem.

$F_1 = \bar{x} \cdot y \cdot \bar{z} + \bar{x} \cdot \bar{y} \cdot z$

$\bar{F}_1 = \overline{\bar{x} \cdot y \cdot \bar{z} + \bar{x} \cdot \bar{y} \cdot z} = \overline{(\bar{x} \cdot y \cdot \bar{z})} \cdot \overline{(\bar{x} \cdot \bar{y} \cdot z)}$

$= (\bar{\bar{x}} + \bar{y} + \bar{\bar{z}}) \cdot (\bar{\bar{x}} + \bar{\bar{y}} + \bar{z}) = \boxed{(x + \bar{y} + z) \cdot (x + y + \bar{z})}$

$F_2 = x \cdot (y \cdot \bar{z} + y \cdot z)$

$\bar{F}_2 = \overline{x \cdot (y \cdot \bar{z} + y \cdot z)} = \bar{x} + (\bar{y} + \bar{\bar{z}} \cdot \bar{y} + \bar{z})$

$\bar{F}_2 = \boxed{\bar{x} + (\bar{y} + z) \cdot (\bar{y} + \bar{z})}$

---

→ Canonical Form :-

1) Min Term & Max Term:

→ A Binary variable may appear either in normal form ($x$) or its compliment form ($\bar{x}$).

⇒ Now, Consider two B. Values $x$ & $y$ combined with an AND operation, since each var. may appear in either, There are four possible Combo :- $x \cdot y$, $\bar{x} \cdot \bar{y}$, $\bar{x} \cdot y$, $x \cdot \bar{y}$.
$(1\ 1)$ $(0\ 0)$ $(0\ 1)$ $(1\ 0)$

Here,
1 → Unbarred
0 → Barred.

each of these four terms are AND terms, Minterm or Std. product.

Symbol :- $m$.

⇒ Similarly, those values combined with an OR operation.

These are the four possible Combo :- $x+y$, $\bar{x}+y$, $x+\bar{y}$, $\bar{x}+\bar{y}$
$(0\ 0)$ $(1\ 0)$ $(0\ 1)$ $(1\ 1)$

Here,
1 → Barred
0 → Unbarred

each of these four terms are OR tems, Maxterm or Std. sum.

| Variables | | | Min term | | Max term | |
|---|---|---|---|---|---|---|
| | | | Term | | Term | |
| $x$ | $y$ | $z$ | | | | |
| 0 | 0 | 0 | $\bar{x}\cdot\bar{y}\cdot\bar{z}$ | $m_0$ | $x+y+z$ | $M_0$ |
| 0 | 0 | 1 | $\bar{x}\cdot\bar{y}\cdot z$ | $m_1$ | $x+y+\bar{z}$ | $M_1$ |
| 0 | 1 | 0 | $\bar{x}\cdot y\cdot\bar{z}$ | $m_2$ | $x+\bar{y}+z$ | $M_2$ |
| 0 | 1 | 1 | $\bar{x}\cdot y\cdot z$ | $m_3$ | $x+\bar{y}+\bar{z}$ | $M_3$ |
| 1 | 0 | 0 | $x\cdot\bar{y}\cdot\bar{z}$ | $m_4$ | $\bar{x}+y+z$ | $M_4$ |
| 1 | 0 | 1 | $x\cdot\bar{y}\cdot z$ | $m_5$ | $\bar{x}+y+\bar{z}$ | $M_5$ |
| 1 | 1 | 0 | $x\cdot y\cdot\bar{z}$ | $m_6$ | $\bar{x}+\bar{y}+z$ | $M_6$ |
| 1 | 1 | 1 | $x\cdot y\cdot z$ | $m_7$ | $\bar{x}+\bar{y}+\bar{z}$ | $M_7$ |

Note:- Each Max term is complementary of its corresponding Min term and vice-versa.

---

→ Sum of products :- A sum of product expression is a product term (min term) or several product terms (min term) logically added (OR) together.

Σ ← (OR) of (AND)

for eg:- the expression $x\cdot\bar{y}+\bar{x}\cdot y$ is sum of product expression

Steps to express a boolean func$^n$ in its sum of product form.

1) → Construct a Truth Table for given boolean func$^n$
2) Find all the combination where value of F=1 ↗ if 1 ⇒ F ↘ if 0 ⇒ F
3) Combination having F=1 are all the min terms obtained

| $x$ | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
|---|---|---|---|---|---|---|---|---|
| $y$ | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| $z$ | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| $F$ | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 1 |
| | $m_0$ | $m_1$ | $m_2$ | $m_3$ | $m_4$ | $m_5$ | $m_6$ | $m_7$ |

eg:- $F=\bar{x}\cdot\bar{y}\cdot z+x\cdot\bar{y}\cdot\bar{z}+x\cdot y\cdot z$
$(0\ 0\ 1)\ (1\ 0\ 0)\ (1\ 1\ 1)$

$F=m_1+m_4+m_7$

$F(x,y,z)=\Sigma(1,4,7)$

→ **Product of Sum :-** A Product of Sum expression is a (AND) (OR) Sumterm (Maxterm) or several sumterm logically multiplied together

for eg:- $(\bar{x}+y)\cdot(x+\bar{y})$ is Product of Sum

Steps to express a boolean func$^n$ in its product of sum form

1) Construct a truth table.
2) find all the combination where value of $F=0$ $\begin{cases} 0 \Rightarrow F \\ 1 \Rightarrow \bar{F} \end{cases}$
3) Comb. having $F=0$ are all the Maxterms obtained

| | x | y | z | F |
|---|---|---|---|---|
| ✓ | 0 | 0 | 0 | 0 |
| | 0 | 0 | 1 | 1 |
| ✓ | 0 | 1 | 0 | 0 |
| ✓ | 0 | 1 | 1 | 0 |
| | 1 | 0 | 0 | 1 |
| ✓ | 1 | 0 | 1 | 0 |
| ✓ | 1 | 1 | 0 | 0 |
| | 1 | 1 | 1 | 1 |

eg:- $F_1 = (x+y+z)\cdot(x+\bar{y}+z)\cdot(x+\bar{y}+\bar{z})\cdot(\bar{x}+y+\bar{z})\cdot(\bar{x}+\bar{y}+z)$.

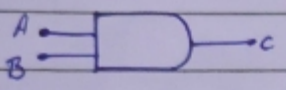
$F_1 = M_0 \cdot M_2 \cdot M_3 \cdot M_5 \cdot M_6$

$F_1(x,y,z) = \pi\,(0,2,3,5,6)$

↳ Basic Building Blocks of Digital Circuits.

**Logic Gates:-** They are electronic circuits that performs operations on one or more input signals to produce a single output signal.

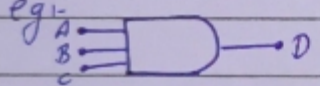
**I) AND Gates:-** Physical realisation of logical mul. operation. It is an electronic circuits that generates output signal 1 only-if all Input signals are 1.




$C = A \cdot B$

| A | B | C |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 ✓ |

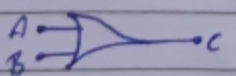
eg:-




$D = A \cdot B \cdot C$

| A | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 ✓ |
|---|---|---|---|---|---|---|---|---|
| B | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| C | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| D | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |

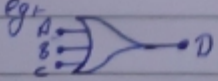
**II) OR Gate:-** P. Realisation of logical addition operation. It is an e-circuit that generates output signal 1 even-if-only one of Input signal is 1.




$C = A + B$

| A | B | C |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 ✓ |
| 1 | 0 | 1 ✓ |
| 1 | 1 | 1 ✓ |

eg:-

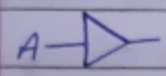



$D = A + B + C$

| A | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
|---|---|---|---|---|---|---|---|---|
| B | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| C | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| D | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |

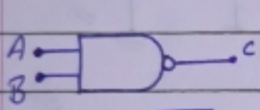
III) NOT Gate :- P.R of compliment operation. e-circuit that generates output signal opposite/reverse of Input signal.

→ Also called inverter as it inverses the output



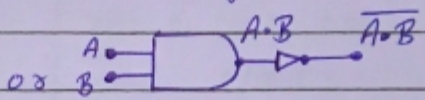

| A | $\bar{A}$ |
|---|---|
| 0 | 1 |
| 1 | 0 |

IV) NAND Gate :- Compliment of "AND" Gate. It shows inverse value of AND Gate.



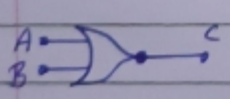

$C = \overline{A \cdot B}$

or



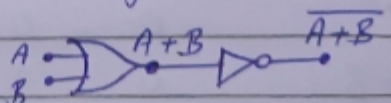

| A | B | A·B | $\overline{A \cdot B}$ |
|---|---|---|---|
| 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |

V) NOR Gate :- Compliment of "OR" Gate. It shows inverse value of OR Gate.



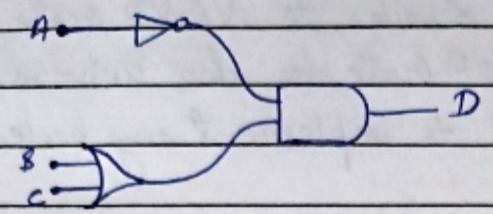

$C = \overline{A + B}$

or



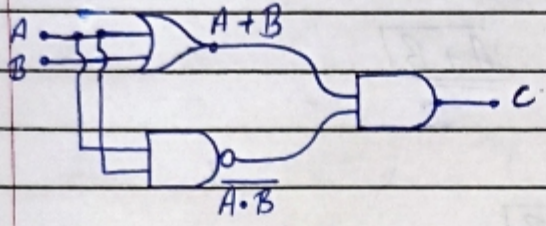

| A | B | A+B | $\overline{A+B}$ |
|---|---|---|---|
| 0 | 0 | 0 | 1 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 0 |

Q




Ans $\boxed{D = \bar{A} \cdot B + C}$

Q.

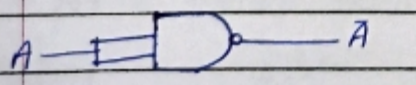



Ans $\boxed{C = (A+B) \cdot (\overline{A \cdot B})}$

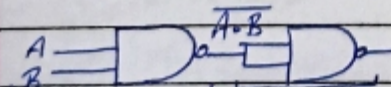
# Universal NAND Gate:- AND, OR, & NOT Gate are logically complete in the sense that any boolean func$^n$ maybe constructing using these gates.

→ The NAND Gate is said to be Universal Gate b/c it is alone sufficient to implement any boolean func$^n$, so that any B. func$^n$ can be implemented with the only use of NAND Gate.

→ We need only to show that logical operation AND, OR can be implemented with NAND Gate.

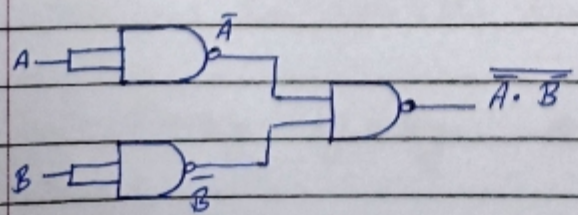



$\overline{A \cdot A} = \bar{A} + \bar{A} = \boxed{\bar{A}}$ $(x + x = x)$



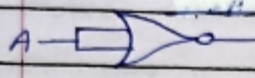

NAND NOT

$= \boxed{A \cdot B}$

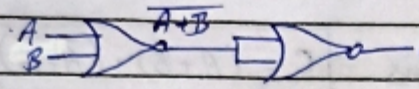



$= \bar{\bar{A}} + \bar{\bar{B}} = \boxed{A + B}$

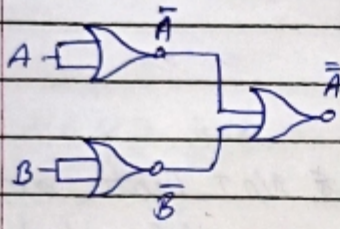
# Universal NOR Gate :- Similar to NAND Gate, NOR Gate is also universal b/c it is alone sufficient to implement any boolean func$^n$.




$\overline{A+A} = \bar{A} \cdot \bar{A} = \boxed{\bar{A}}$ $(x \cdot x = x)$




$\boxed{\overline{A+B}}$

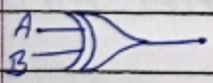



$= \boxed{A \cdot B}$

# Exclusive OR Gate :- Commonly written as (Ex-OR) or (XOR)

→ Output is 1 when all input are unlike, and logic 0 when like. [XOR is an odd func$^n$, = 1 if input var. have odd no. of 1s.]

→ The output is shown as :-

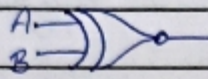



$y = A \oplus B = \bar{A} \cdot B + A \cdot \bar{B}$

| A | B | y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

PAGE NO.:

# Ex-NOR :- It is also written as $\dot{X}NOR$, (Compliment of XOR) means NOT of XOR gate.




$$y = \overline{A \oplus B} = \overline{\bar{A} \cdot B + A \cdot \bar{B}} = \overline{\bar{A} \cdot B} \cdot \overline{A \cdot \bar{B}}$$

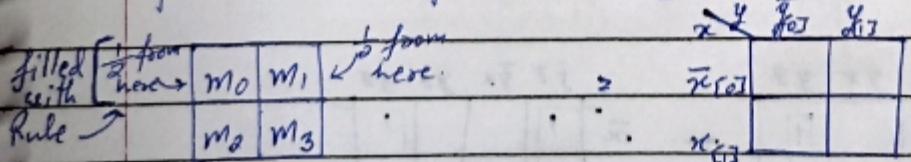
$$= \underline{\underline{A + \bar{B} \cdot A + \bar{B}}}$$

| A | B | y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

# Karnaugh Map (K-Map) :- It is a st. forward method for minimizing Boolean func$^n$. Pictorial form of Truth Table.

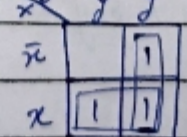
→ It consists of squares, where each square represents one minterm of the func$^n$. By identifying patterns in the map, we can obtain alternative Boolean expression and select the simplest one.

→ for a Boolean func$^n$ with $\underline{n}$ variables, the number of cells = $\underline{\underline{2^n}}$




This was Two variable K-map.

eg:- $F = \bar{x}y + x\bar{y} + xy$ ⇒



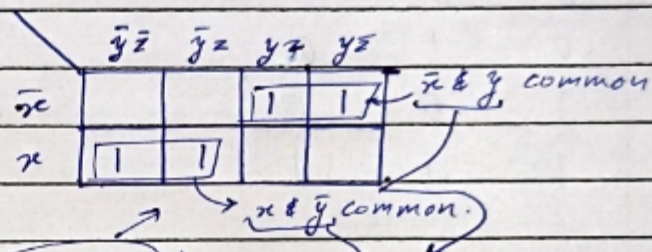

$F = x + y$.

→ Rules of creating Groups in K-map :-

1) NO Zero "0" allowed.
2) Grps can be Vertical or Horizontal but can't be diagonals.
3) Overlapping allowed.
4) Grps size should be in "$2^n$" form ⇔ 2, 4, 8 & it should contain $2^n$ cells.
5) Grps may only consist of 2, 4, 8, 16 .....

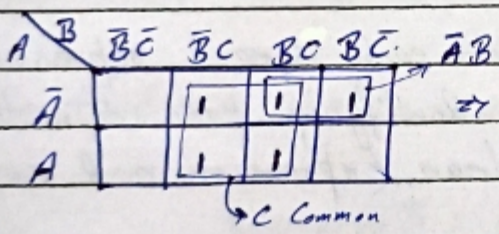
→ 3 Var. K-map :-

| | | | |
|---|---|---|---|
| $m_0$ | $m_1$ | $m_3$ | $m_2$ |
| $m_4$ | $m_5$ | $m_7$ | $m_6$ |



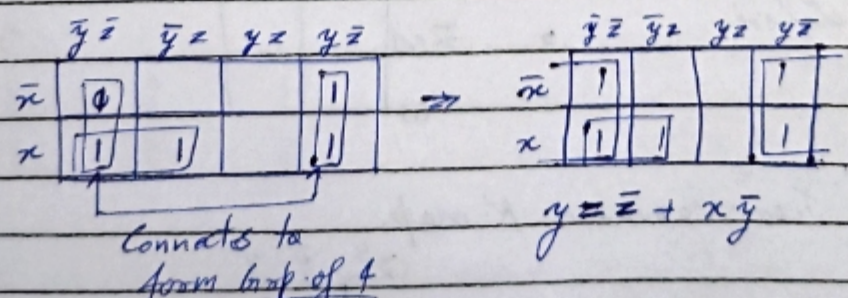

eg :- $F(x,y,z) = \Sigma(2,3,4,5) = x\bar{y} + \bar{x}y$

eg :- $y = A \cdot B \cdot C + A \cdot \bar{B} \cdot C + \bar{A} \cdot B \cdot \bar{C} + \bar{A} \cdot \bar{B} \cdot C + \bar{A} \cdot B \cdot C$




⇒ $y = C + \bar{A}B$

eg :- $F(x,y,z) = \Sigma(0,2,4,5,6)$.



→ 4 Var. K-map:-

| | | | | |
|---|---|---|---|---|
| | 0 | 1 | 3 | 2 |
| | 4 | 5 | 7 | 6 |
| | 12 | 13 | 15 | 14 |
| | 8 | 9 | 11 | 10 |

| | $\bar{y}\bar{z}$ | $\bar{y}z$ | $yz$ | $y\bar{z}$ |
|---|---|---|---|---|
| $w\bar{x}$ | | | | |
| ⇒ $\bar{w}x$ | | | | |
| $wx$ | | | | |
| $w\bar{x}$ | | | | |

eg:- $F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$

| | $\bar{y}\bar{z}$ | $\bar{y}z$ | $yz$ | $y\bar{z}$ |
|---|---|---|---|---|
| $\bar{w}\bar{x}$ | 1 | 1 | | 1 |
| $\bar{w}x$ | 1 | 1 | | 1 |
| $wx$ | 1 | 1 | | 1 |
| $w\bar{x}$ | 1 | 1 | | |

$y = \bar{y} + \bar{w}\bar{z} + x\bar{z}$

→ Some Looping variables :-

| | | | |
|---|---|---|---|
| | | | 1 |
| 1 | | 1 | |

| | | | |
|---|---|---|---|
| | 1 | | |
| | | | |
| | | | |
| | 1 | | |

| | | | |
|---|---|---|---|
| | 1 | 1 | |
| | | | |
| | | | |
| 1 | 1 | | |

| | | | |
|---|---|---|---|
| 1 | | | 1 |
| | | | |
| | | | |
| 1 | | | 1 |

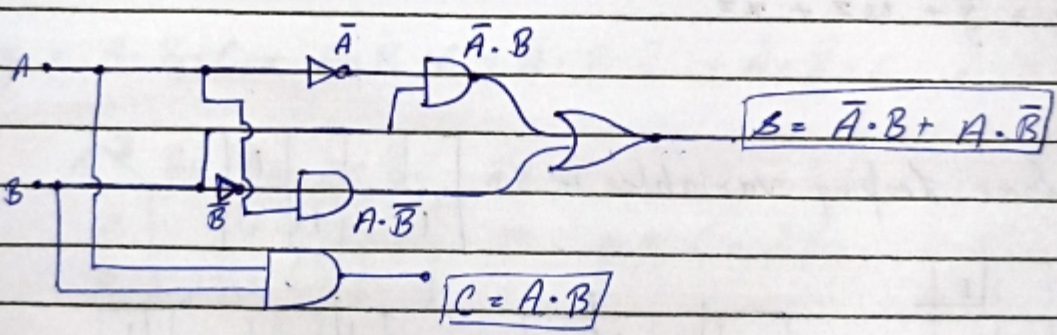
# Design of Half-Adder :- This circuit needs two B. Inputs and 2 B. Output, the input var. designates the Augend & addend bits, there output var. produced the same and carry bits.

Let A & B be 2 inputs, S (for Sum) an C (for Carry) be the 2 outputs

| Input | | Output | |
|---|---|---|---|
| A | B | C | S |
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |

$S = \bar{A} \cdot B + A \cdot \bar{B}$

$C = A \cdot B$

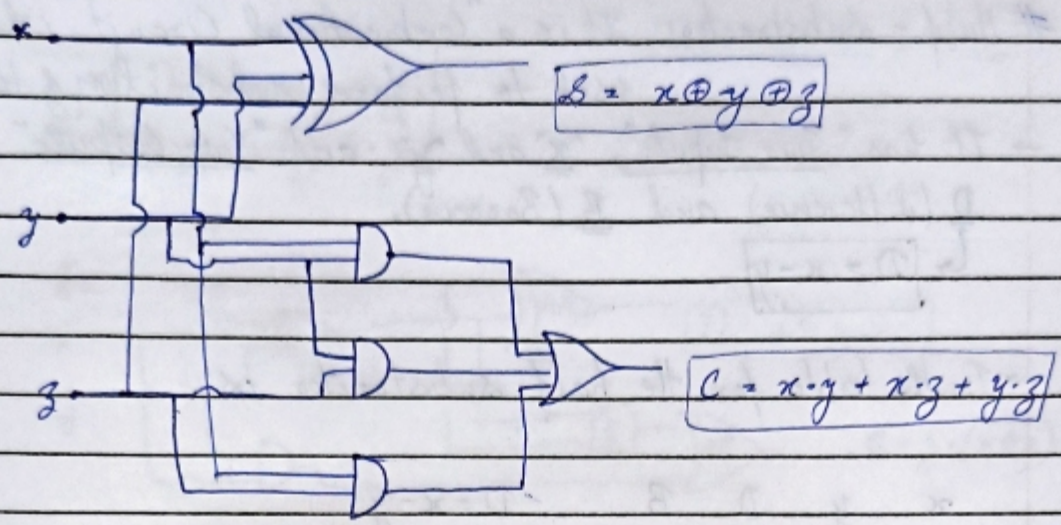



# Design of full-adder :- It forms Arithmetic Sum of 3 Input Bits thus it contains of 3 Inputs and 2 Outputs. 2 of the Input var. represents the Augend and Addend bits and the 3rd Input var. represents the carry from the previous lower significant positions.

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| I | A | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| | B | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| | C | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| O | Ca | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 1 |
| | S | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 1 |

$Sum \Rightarrow x \oplus y \oplus z$

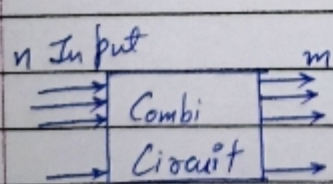
$C \Rightarrow xy + xz + yz$

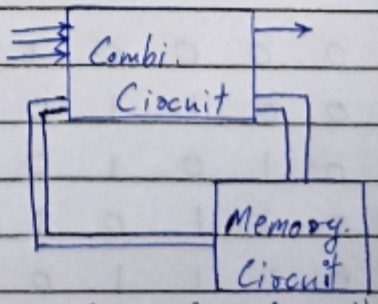



⇒ Type of Logic Gate :- Logic Circuits for Digital System, may be Combinational or Sequential.

① Combinational Circuits :- It consists of L. Gates whose Output at any time are determined from the present combination of Inputs a combination circuits consists of I. var., L. Gates, & Output var., basic combi. building blocks are logic Gates.

② Sequential Circuits :- It consists of Combinational Circuit to which storage element (memory) are connected to form a feed back path, the input path together with the present stage of the storage element, determines the B. value of output Counter is a Sequential circuit.




Sequential Circuit

# Half - Substractor :- It is a "Combinational Circuit" which is used to perform substraction of two bits.

→ It has "Two Inputs", "x" and "y" and "Two Outputs" D (difference) and B (Borrow).

↳ $D = x - y$

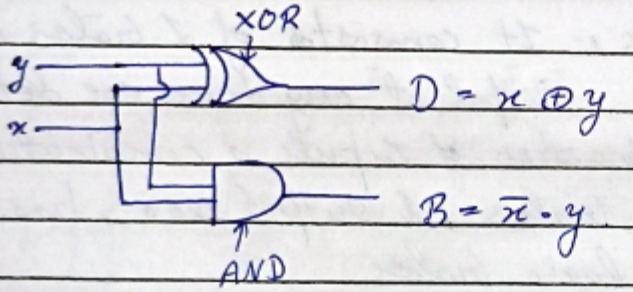
→ Truth table for the half substractor is :-

| x | y | D | B |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 |

$D = x - y$

eq" $D = x \oplus y$

$B = \bar{x} \cdot y$

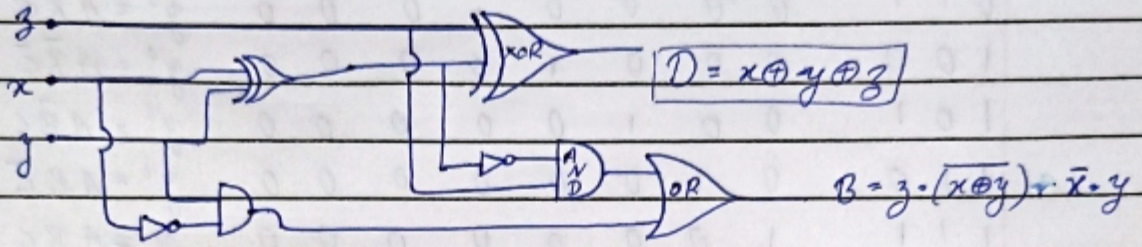



# Full - Substractor :- It is a "Combinational Circuit" which is used to perform substraction of three bits, x and y and z. and Two Outputs 'D' (Difference) and 'B' (Borrow.

→ Truth Table :- $D = x - y - z$

| x | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
|---|---|---|---|---|---|---|---|---|
| y | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| z | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| D | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 1 |
| B | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 1 |

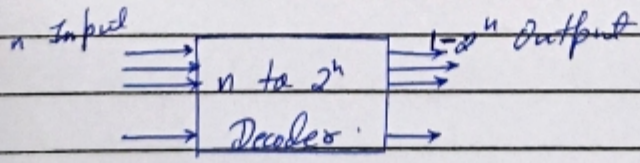
eq$^n$ $\begin{bmatrix} D = x \oplus y \oplus z \\ B = z \cdot (\overline{x \oplus y}) + \bar{x} \cdot y \end{bmatrix}$



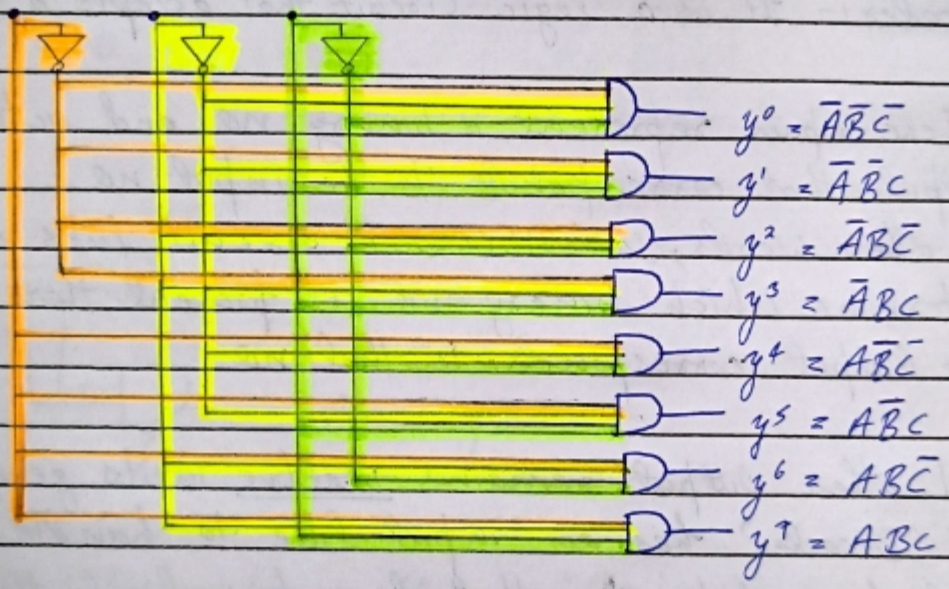

# Decoder :- It is a logic Circuit that accepts a "set of Inputs".

→ These inputs represent a binary no. and activates only the output that corresponds to the input no.
→ In other words, a 'DECODER' circuit looks at it's input, determine which binary no. is present there and activates one Output corresponds to that no.

→ All other output remains inactive in its general form
→ A 'Decoder' has an input line to handle a 'N bits' and form 1 to $2^n$ output lines to indicate the presence of one or more "N bits combinations".

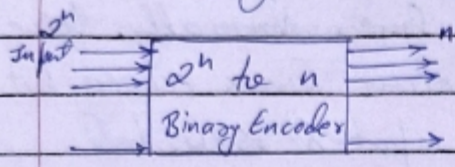


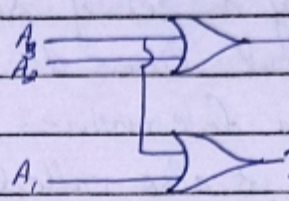
| A | B | C | $y_7$ | $y_6$ | $y_5$ | $y_4$ | $y_3$ | $y_2$ | $y_1$ | $y_0$ | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | $y^0 = \bar{A}\bar{B}\bar{C}$ |
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | $y^1 = \bar{A}\bar{B}C$ |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | $y^2 = \bar{A}B\bar{C}$ |
| 0 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | $y^3 = \bar{A}BC$ |
| 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | $y^4 = A\bar{B}\bar{C}$ |
| 1 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | $y^5 = A\bar{B}C$ |
| 1 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | $y^6 = AB\bar{C}$ |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | $y^7 = ABC$ |



PAGE NO :

# Encoder :- It is a combinational circuit that performs the inverse operation of a decoder, if a drive output code has a fewer bits than the input code has, the device is usually called encoder.



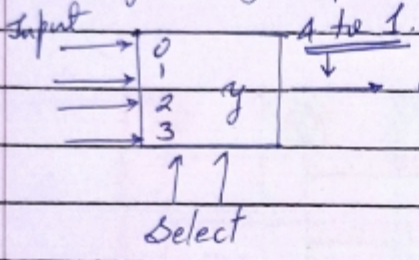

$y_0 = A_1 + A_3$ ; $y_1 = A_2 + A_3$

| Inputs | | | | Outputs | |
|---|---|---|---|---|---|
| $A_3$ | $A_2$ | $A_1$ | $A_0$ | $y_1$ | $y_0$ |
| 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 | ① → $A_1 + A_3$ |
| 0 | 1 | 0 | 0 | ① | 0 |
| 1 | 0 | 0 | 0 | ① → $A_2 + A_3$ | ① |

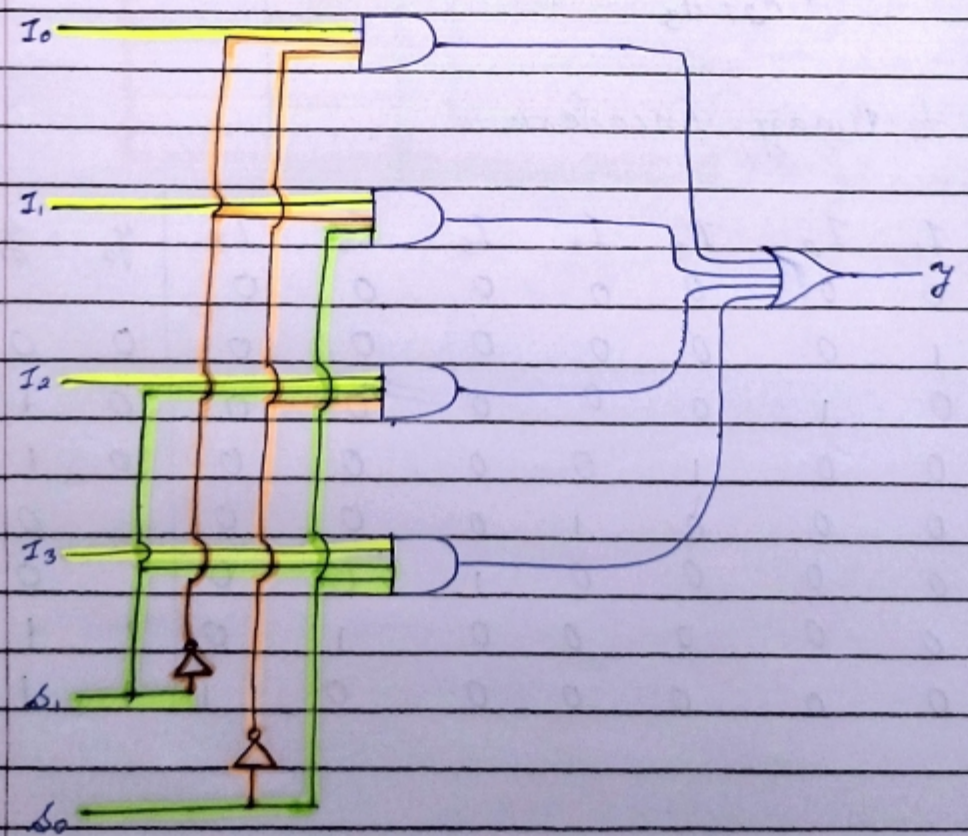
Octal to Binary Encoder :-

| $I_0$ | $I_1$ | $I_2$ | $I_3$ | $I_4$ | $I_5$ | $I_6$ | $I_7$ | $y_2$ | $y_1$ | $y_0$ |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | | | |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |

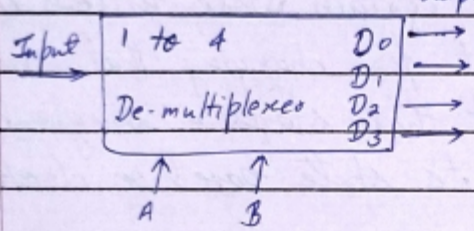
# Multiplexers :- A digital multiplexer is a combinational circuit that select binary information from one to many input lines & directs it to a single output line. The selection of particular input line is controlled by a set of selection line. Normally, there are $2^n$ input lines & n selection lines whose bit combination determines which input is selected. Multiplexer is also called data selector, since it select one of many input a 4 to 1 multiplexer is shown below.



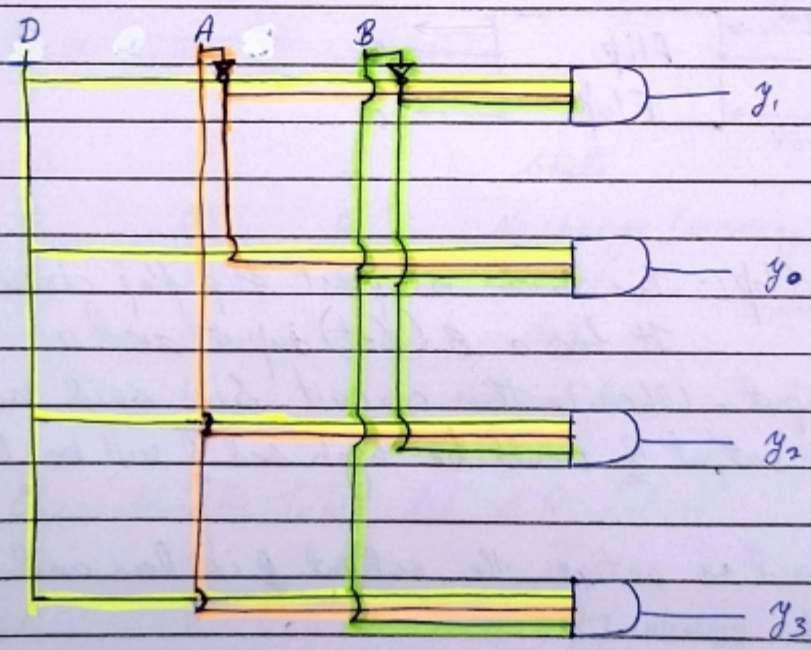

| $S_1$ | $S_0$ | y |
|---|---|---|
| 0 | 0 | $I_0$ |
| 0 | 1 | $I_1$ |
| 1 | 0 | $I_2$ |
| 1 | 1 | $I_3$ |



# De-Multiplexer :- It is combinational logic circuit with an input line, $2^n$, output lines and $n$ selection lines, it transfer information present on the input line to any of the output line, the output line that guess information present on the input line is the decided by the selection lines following figures shows a 1 to 4 de-multiplexer.

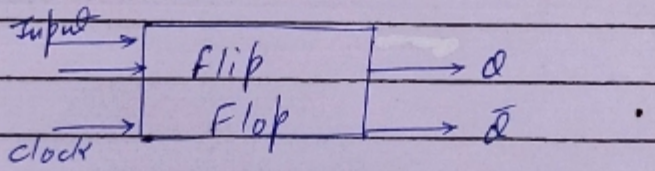



| Input | Selected | | Output | | | |
|---|---|---|---|---|---|---|
| | A | B | $y_0$ | $y_1$ | $y_2$ | $y_3$ |
| 1 | 0 | 0 | 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 | 1 | 0 |
| 1 | 1 | 1 | 0 | 0 | 0 | 1 |



# Flip-Flop:- It is a circuit that maintains a state until directed by input to change the state a basic flip-flop can be constructed using 4 NAND Gate or NOR Gate. Flip-flop is popularly known as the basic digital memory circuit. It has two states logic one or logic zero. & They are fundamental building blocks of all memory devices.

The Flip-Flop maintains a certain state unless and until directed by the input for changing that state. It is sequential logic circuit that stores a single bit (0 or 1) of binary data and change its state based on clock signal.

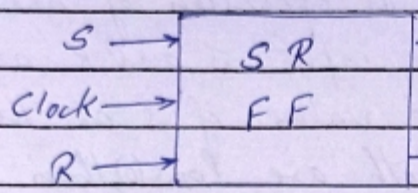


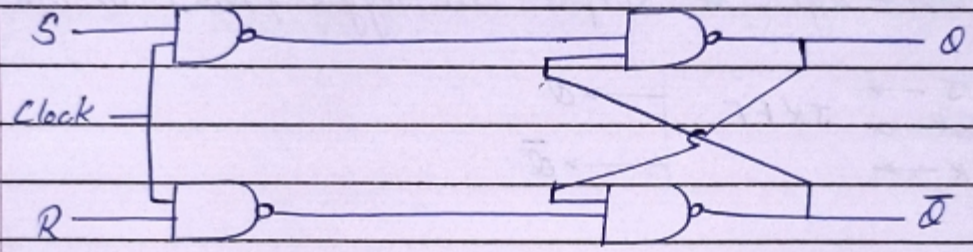

→ S-R Flip Flop:- This is the simplest flip-flop circuit. It has a S (Set) input and a R (Reset) input. When in this circuit S is said as active, the output Q would be high and $\bar{Q}$ will be low.

if R is said as active the output Q is low and $\bar{Q}$ is high

Once the output are established, the results of the circuit are maintained, until S or R get changed or the power is turned off when Q=1 & $\bar{Q}$=0, the flip-flop is said to be in the set state.
When Q=0 & $\bar{Q}$=1; it is in the reset state.

⇒ Output Q and $\bar{Q}$ are normally be compliment of each other when both inputs are 1 at the same time, and undefined state with both output equal occurs.



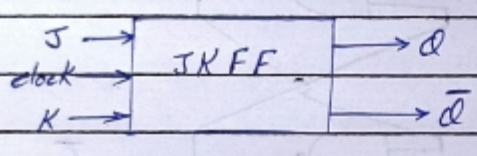

Truth Table of S-R F.F:-

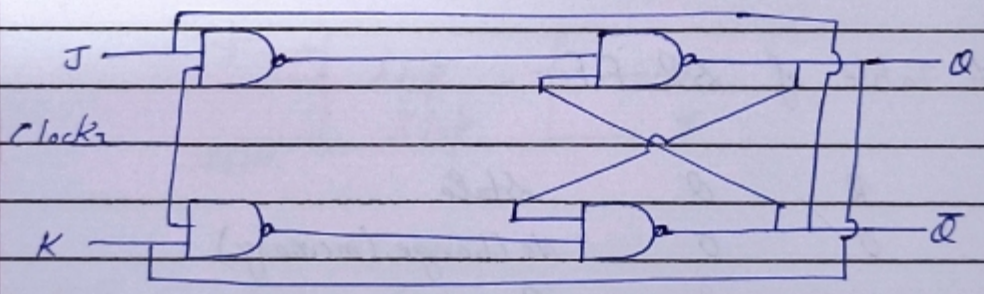
| S | R | Q | State |
|---|---|---|---|
| 0 | 0 | 0 | No change (memory) |
| 0 | 1 | 0 | Reset |
| 1 | 0 | 1 | Set |
| 1 | 1 | x | Invalid |

Characteristic Table for S-R F-F:-

| S | R | $Q_{(t)}$ | $Q_{(t+1)}$ | |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | memory |
| 0 | 0 | 1 | 1 | |
| 0 | 1 | 0 | 0 | Reset |
| 0 | 1 | 1 | 0 | |
| 1 | 0 | 0 | 1 | Set |
| 1 | 0 | 1 | 1 | |
| 1 | 1 | 0 | x | Invalid |
| 1 | 1 | 1 | x | |

# J.K. F.F. :- Because of the Invalid state corresponding to S=R=1 in SRFF, there is a need of another flip-flop. The JKFF operates with only +ve or -ve clock transition, the operation of JKFF is similar to the SRFF. When the input J and K are different, then the output Q takes the value of J at the next clock age. When J and K both are low(0), then no change occur at the output, if J and K are 1, then at the clock age, the output will toggle from 1 to other.




Truth table :-

| J | K | Q | State. |
|---|---|---|---|
| 0 | 0 | 0 | No Change |
| 0 | 1 | 0 | Reset |
| 1 | 0 | 1 | Set |
| 1 | 1 | x | Toggle Toggle. |

Characteristic table for JKFF :-

| S | R | $Q_{(t)}$ | $Q_{(t+1)}$ | |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | No change |
| 0 | 0 | 1 | 1 | |
| 0 | 1 | 0 | 0 | Reset |
| 0 | 1 | 1 | 0 | |
| 1 | 0 | 0 | 1 | Set |
| 1 | 0 | 1 | 1 | |
| 1 | 1 | 0 | x | Invalid. |
| 1 | 1 | 1 | x | |